

Name: Sameer Almadani

Student ID: 2230001818

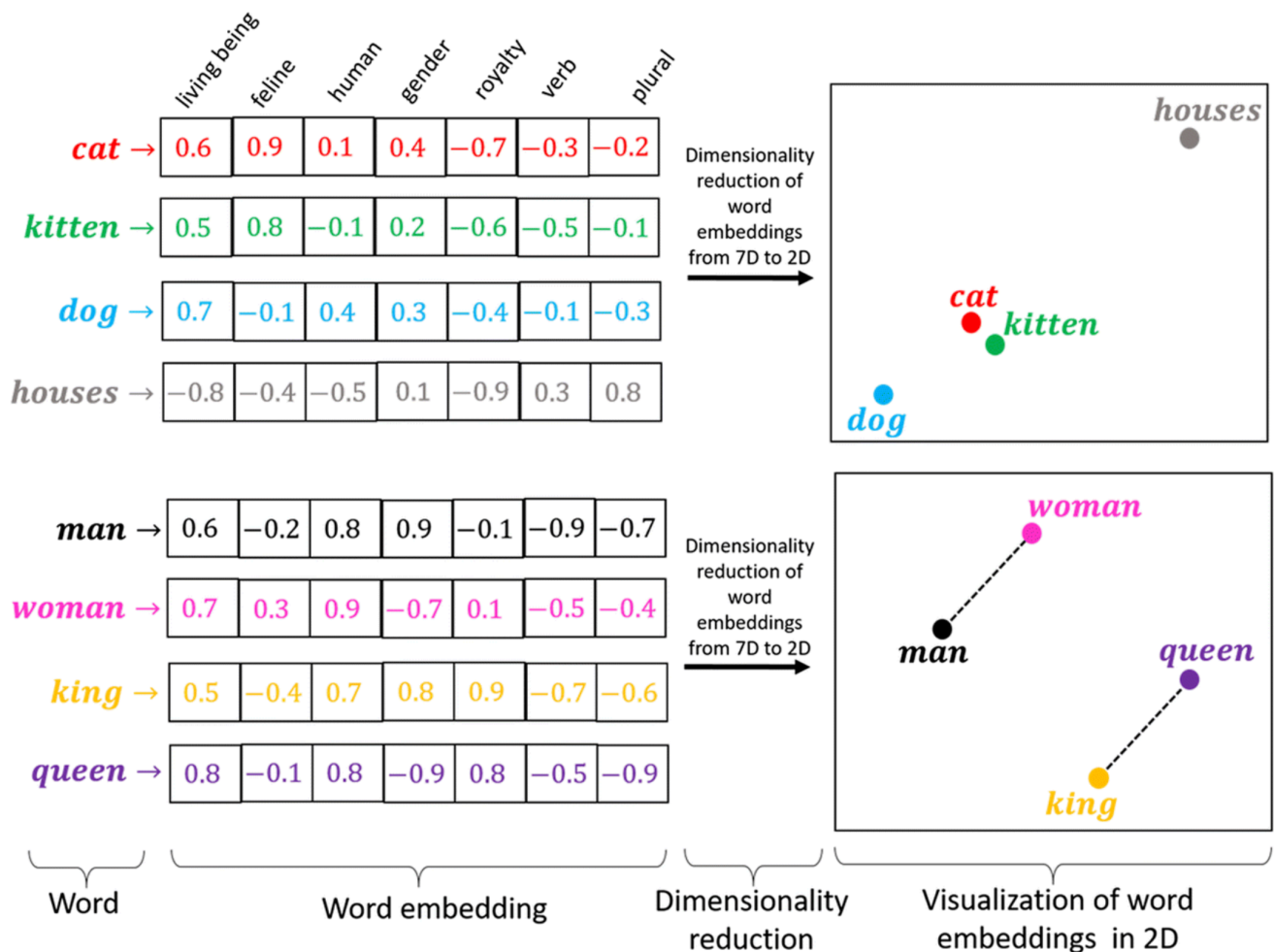
Text Representation

Text representation in NLP means converting text into a numerical form that a machine learning model can understand and process.

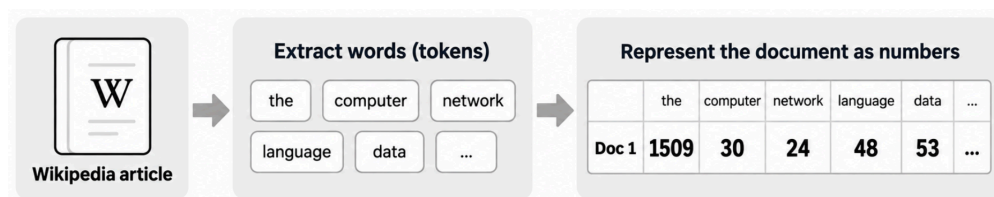
What is Embedding?

Embeddings in NLP is a technique where individual words are represented as real-valued vectors and captures inter-word semantics.

In this notebook, We going to introduce 2 techniques for embedding. These techniques will be used for a machine learning models such as SVM, Random forest, ... ect.



1- TF-IDF



TF-IDF stands for term frequency-inverse document frequency. It is a measure that discounts common words. Used in the fields of information retrieval (IR) and machine learning, that can quantify the importance of words in a document amongst a collection of documents (also known as a corpus).

Components of TF-IDF

1. TF (Term Frequency):

Measures how frequently a term appears in a document.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

2. IDF (Inverse Document Frequency):

Measures how important a term is across all documents. Words that appear in many documents get lower scores.

$$IDF(t) = \log \left(\frac{\text{Number of all documents } N}{\text{Number of documents containing the term } t} \right)$$

	...	the	country	hell	...
...					
425		43	5	1	
426		24	1	0	
427		37	0	3	
...					
df		500	14	7	

tf matrix

	...	the	country	hell	...
...					
425		0	26.0	6.2	
426		0	5.2	0	
427		0	0	18.6	
...					

tf-idf matrix

12

```
%pip install pandas scikit-learn

from sklearn.feature_extraction.text import TfidfVectorizer
```

Requirement already satisfied: pandas in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (3.0.0)

Requirement already satisfied: scikit-learn in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (1.8.0)

Requirement already satisfied: numpy>=1.26.0 in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
pandas) (2.4.0)

Requirement already satisfied: python-dateutil>=2.8.2 in
/Users/almadani/Library/Python/3.13/lib/python/site-packages (from pandas)
(2.9.0.post0)

Requirement already satisfied: scipy>=1.10.0 in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
scikit-learn) (1.17.1)

Requirement already satisfied: joblib>=1.3.0 in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
scikit-learn) (1.5.3)

```
Requirement already satisfied: threadpoolctl>=3.2.0 in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
scikit-learn) (3.6.0)
Requirement already satisfied: six>=1.5 in
/Users/almadani/Library/Python/3.13/lib/python/site-packages (from python-
dateutil>=2.8.2->pandas) (1.17.0)
```

[notice] A new release of pip is available: 25.2 -> 26.2.1

[notice] To update, run: /Users/almadani/Desktop/study/nlp/nlp lab/.venv/bin/python -m
pip install --upgrade pip

Note: you may need to restart the kernel to use updated packages.

```
doc_1 = "Data is the oil of the digital economy"
doc_2 = "Data is a new oil"

data = [doc_1, doc_2]
```

```
tfidf = TfidfVectorizer()
result = tfidf.fit_transform(data) # returns sparse matrix
```

```
import pandas as pd

df = pd.DataFrame(result.toarray(), columns=tfidf.get_feature_names_out())
df
```

	data	digital	economy	is	new	of	oil	the
0	0.243777	0.34262	0.34262	0.243777	0.000000	0.34262	0.243777	0.68524
1	0.448321	0.00000	0.00000	0.448321	0.630099	0.00000	0.448321	0.00000

Cosine similarity

In NLP, Cosine similarity is a metric used to measure how similar the documents are.

$$\text{cosine similarity} = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Where:

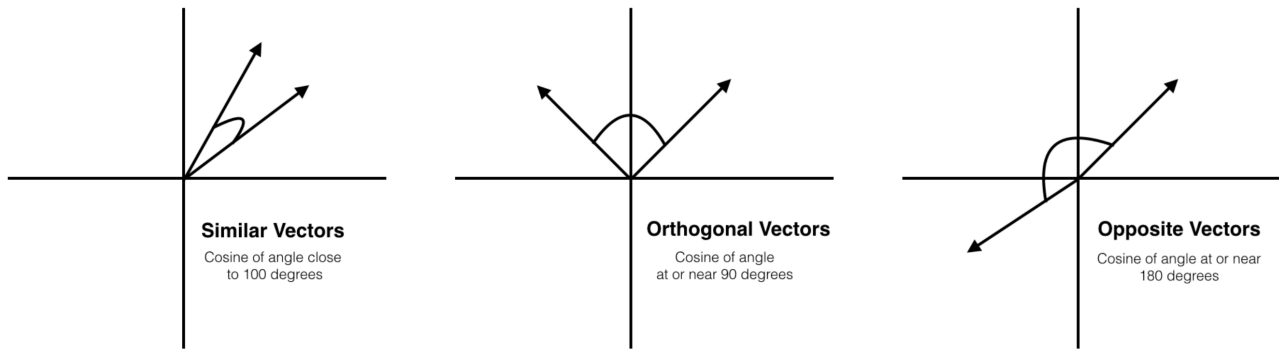
$A \cdot B$ = dot product of vectors A and B

$\|A\|$ = magnitude (length) of vector A

$\|B\|$ = magnitude (length) of vector B

Intuition

- If vectors point in the **same direction**, cosine similarity = **1** (maximum similarity).
- If vectors are **orthogonal (90° apart)**, cosine similarity = **0** (no similarity).



```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
# Sample text data (replace with your own documents)
```

```
doc_1 = "Data is the oil of the digital economy"
doc_2 = "Data is a new oil"
```

```
data = [doc_1, doc_2]
```

```
tfidf_vectorizer = TfidfVectorizer() # Create a CountVectorizer instance
vector_matrix = tfidf_vectorizer.fit_transform(data) # Fit and transform the documents into
numerical vectors
```

```
# Calculate the cosine similarity between the documents
cosine_similarity_matrix = cosine_similarity(vector_matrix)

df_cosine = pd.DataFrame(data=cosine_similarity_matrix, index=data, columns=data)

df_cosine
```

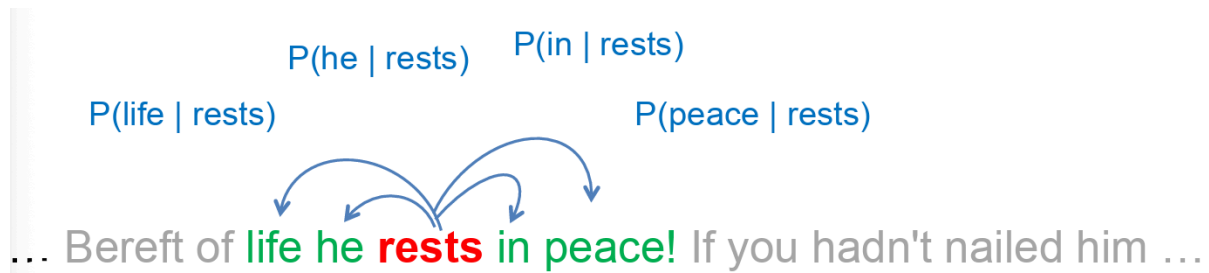
	Data is the oil of the digital economy	Data is a new oil
Data is the oil of the digital economy	1.000000	0.327871
Data is a new oil	0.327871	1.000000

2- What is Word2vec?

Word2Vec consists of models for generating word embedding. These models are two-layer neural networks having one input layer, one hidden layer, and one output layer.

Word2Vec utilizes two architectures :

1. CBOW (Continuous Bag of Words)
2. **Skip Gram**



Run this command in terminal to install

```
pip install gensim
```

We will use fake and real news dataset to do our experiment. You can find the dataset here:
<https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset?select=True.csv>
<https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset?select=True.csv>

```
%pip install nltk gensim certifi

import pandas as pd
import nltk
import os
import certifi
os.environ.setdefault('SSL_CERT_FILE', certifi.where())
nltk.download('punkt')
nltk.download('punkt_tab')
import numpy as np
import gensim

from nltk.tokenize import word_tokenize
```

```
Requirement already satisfied: nltk in /Users/almadani/Desktop/study/nlp/nlp
lab/.venv/lib/python3.13/site-packages (3.10.3)
Requirement already satisfied: gensim in /Users/almadani/Desktop/study/nlp/nlp
lab/.venv/lib/python3.13/site-packages (4.4.0)
Requirement already satisfied: certifi in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages
(2026.5.20)
Requirement already satisfied: defusedxml in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
nltk) (0.7.1)
Requirement already satisfied: click in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
nltk) (8.4.2)
Requirement already satisfied: joblib in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in
/Users/almadani/Library/Python/3.13/lib/python/site-packages (from nltk) (2026.7.19)
Requirement already satisfied: tqdm in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
nltk) (4.67.3)
Requirement already satisfied: numpy>=1.18.5 in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
gensim) (2.4.0)
Requirement already satisfied: scipy>=1.7.0 in
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (from
gensim) (1.17.1)
Requirement already satisfied: smart_open>=1.8.1 in
/Users/almadani/Desktop/study/nlp/nlp lab/.venv/lib/python3.13/site-packages (from
gensim) (8.0.1)
```

Requirement already satisfied: wrapt in /Users/almadani/Desktop/study/nlp/nlp
lab/.venv/lib/python3.13/site-packages (from smart_open>=1.8.1->gensim) (2.4.1)

[notice] A new release of pip is available: 25.2 -> 26.2.1

[notice] To update, run: /Users/almadani/Desktop/study/nlp/nlp lab/.venv/bin/python -m
pip install --upgrade pip

Note: you may need to restart the kernel to use updated packages.

```
[nltk_data] Downloading package punkt to /Users/almadani/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data] /Users/almadani/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

```
df = pd.read_csv('True.csv', engine='python', on_bad_lines='skip') #fake-and-real-news-dataset-true.csv
df.head()
```

	title	text	subject	date
0	As U.S. budget fight looms, Republicans flip t...	WASHINGTON (Reuters) - The head of a conservat...	politicsNews	December 31, 2017
1	U.S. military to accept transgender recruits o...	WASHINGTON (Reuters) - Transgender people will...	politicsNews	December 29, 2017
2	Senior U.S. Republican senator: 'Let Mr. Muell...	WASHINGTON (Reuters) - The special counsel inv...	politicsNews	December 31, 2017
3	FBI Russia probe helped by Australian diplomat...	WASHINGTON (Reuters) - Trump campaign adviser ...	politicsNews	December 30, 2017
4	Trump wants Postal Service to charge 'much mor...	SEATTLE/WASHINGTON (Reuters) - President Donal...	politicsNews	December 29, 2017

```
tokens = []

for i in df['text']:
    token = word_tokenize(i)
    tokens.append(token)
```

```
w2v = gensim.models.Word2Vec(tokens, min_count=1, vector_size=100, window=5, sg=1)
```

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'

```
print("Cosine similarity between 'provide' and 'program' - Skip Gram :
",w2v.wv.similarity('provide', 'program'))
```

Cosine similarity between 'provide' and 'program' - Skip Gram : 0.4220209

```
print("words that similar to 'program' - Skip Gram : ",w2v.wv.most_similar('program'))
```

```
words that similar to 'program' - Skip Gram : [('programs', 0.8167316317558289),
('programme', 0.8097732663154602), ('programmes', 0.7013993859291077), ('bond-buying',
0.6745232343673706), ('EB-5', 0.6736090183258057), ('entitlement', 0.6733730435371399),
('arsenal', 0.6721699833869934), ('guest-worker', 0.666012167930603), ('terminating',
0.6548282504081726), ('curtail', 0.6542850136756897)]
```

Tasks

Task 1: Cosine Similarity

Use the Cosine Similarity method to determine how similar the following sentences are.

'This is the first document.'
 'This document is the second document.'
 'And this is the third one.'
 'Is this the first document?'

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

sentences = [
    'This is the first document.',
    'This document is the second document.',
    'And this is the third one.',
    'Is this the first document?'
]

vectors = TfidfVectorizer().fit_transform(sentences)
similarities = cosine_similarity(vectors)
pd.DataFrame(similarities,
              index=['Sentence 1', 'Sentence 2', 'Sentence 3', 'Sentence 4'],
              columns=['Sentence 1', 'Sentence 2', 'Sentence 3', 'Sentence 4'])
```

	Sentence 1	Sentence 2	Sentence 3	Sentence 4
Sentence 1	1.000000	0.646926	0.307772	1.000000
Sentence 2	0.646926	1.000000	0.225240	0.646926
Sentence 3	0.307772	0.225240	1.000000	0.307772
Sentence 4	1.000000	0.646926	0.307772	1.000000

Task 2: TF-IDF

Use tf-idf method on the sentences below to determine the important words.

'data science is one of the most important fields of science',
 'this is one of the best data science courses',
 'data scientists analyze data'

```
sentences = [
    'data science is one of the most important fields of science',
    'this is one of the best data science courses',
    'data scientists analyze data'
]

tfidf = TfidfVectorizer()
vectors = tfidf.fit_transform(sentences)
scores = pd.DataFrame(vectors.toarray(), columns=tfidf.get_feature_names_out(),
                      index=['Sentence 1', 'Sentence 2', 'Sentence 3'])
display(scores)
for sentence, row in scores.iterrows():
    print(sentence, ': ', row[row == row.max()].index.tolist())
```

	analy ze	best	cours es	data	fields	import ant	is	most	of	one	scien ce	scient ists	the	this
Senten ce 1	0.000 000	0.000 000	0.000 000	0.189 526	0.320 895	0.3208 95	0.244 049	0.320 895	0.488 098	0.244 049	0.488 098	0.0000 00	0.244 049	0.000 000
Senten ce 2	0.000 000	0.400 294	0.400 294	0.236 420	0.000 000	0.0000 00	0.304 434	0.000 000	0.304 434	0.304 434	0.304 434	0.0000 00	0.304 434	0.400 294
Senten ce 3	0.542 701	0.000 000	0.000 000	0.641 055	0.000 000	0.0000 00	0.000 000	0.000 000	0.000 000	0.000 000	0.000 000	0.5427 01	0.000 000	0.000 000

Sentence 1 : ['of', 'science']

Sentence 2 : ['best', 'courses', 'this']

Sentence 3 : ['data']

Word2vec

Task 3:

Download the Simpsons dataset (**simpsons_script_lines.csv**) and apply the preprocessing procedure. Use the **'spoken_words'** column.

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"[0-9]", '', text)
    text = re.sub(r"[\s\"'.,-]", '', text)
    return text
```

Create a skip gram Word2Vec model as below.

```
Skip_gram_model = gensim.models.Word2Vec(tokens, min_count = 1, vector_size = 100, window = 5,
sg = 1)
```

```
import pandas as pd
import re
import nltk
nltk.download('punkt_tab')
import gensim
from nltk.tokenize import word_tokenize

simpsons = pd.read_csv('simpsons_script_lines.csv')
simpsons = simpsons.dropna(subset=['spoken_words']).copy()

def clean_text(text):
    text = text.lower()
    text = re.sub(r"[0-9]", '', text)
    text = re.sub(r"[\s\"'.,-]", '', text)
    return text

simpsons['spoken_words'] = simpsons['spoken_words'].apply(clean_text)
tokens = [word_tokenize(text) for text in simpsons['spoken_words'] if text.strip()]
Skip_gram_model = gensim.models.Word2Vec(tokens, min_count=1, vector_size=100, window=5, sg=1)
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data] /Users/almadani/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

```
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
```

```
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
```

```
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
```

```
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
```

```
Exception ignored in: 'gensim.models.word2vec_inner.our_dot_float'
```

Task 4

Use: `wv.most_similar()` method to :

1. Find the words similar to “homer”.
2. Find the words similar to “marge”.
3. Find the words similar to “bart”

```
for word in ['homer', 'marge', 'bart']:
    print(word)
    print(Skip_gram_model.wv.most_similar(word))
    print()
```

```
homer
[('abe', 0.8802703619003296), ('marge', 0.8677650094032288), ('eliza',
0.8277429342269897), ('grampa', 0.8274142146110535), ('ned', 0.8270664215087891),
('mindy', 0.8212984800338745), ('barney', 0.8110101819038391), ('bart',
0.810428261756897), ('bartholomew', 0.8076909184455872), ('boob', 0.8074521422386169)]
```

```
marge
[('homer', 0.867764949798584), ('abe', 0.8578233122825623), ('sweetheart',
0.8308906555175781), ('lisa', 0.8223898410797119), ('becky', 0.8146559596061707),
('allison', 0.8106942772865295), ('sweetie', 0.804082989692688), ('jessica',
0.8029566407203674), ('honey', 0.8002969622612), ('apu', 0.8002077341079712)]
```

```
bart
[('lisa', 0.8784756660461426), ('milhouse', 0.8404133915901184), ('jessica',
0.8389543890953064), ('eliza', 0.8275315165519714), ('grampa', 0.8208848834037781),
('abe', 0.8181410431861877), ('sweetheart', 0.8125364184379578), ('allison',
0.8105810284614563), ('homer', 0.810428261756897), ('apu', 0.8087813258171082)]
```

Task 5

Use the `wv.doesnt_match()` method to :

1. Find which of 'jimbo', 'milhouse', and 'kearney' does not belong to the list.
2. Find the odd one among "nelson", "bart", and "milhouse".
3. Find the odd one among 'homer', 'patty', and 'selma'.

Hint: You need to pass the strings as List

```
groups = [
    ['jimbo', 'milhouse', 'kearney'],
    ['nelson', 'bart', 'milhouse'],
    ['homer', 'patty', 'selma']
]

for words in groups:
    print(words, '->', Skip_gram_model.wv.doesnt_match(words))
```

```
['jimbo', 'milhouse', 'kearney'] -> milhouse
['nelson', 'bart', 'milhouse'] -> nelson
['homer', 'patty', 'selma'] -> homer
```